

# 2026-06-10-SP6-new-submodules-design

## SP6 — NEW HelixDevelopment Submodules Design Spec (G-2 decision feeder)

| Field           | Value                                                                                                                                                                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Revision        | 1                                                                                                                                                                                                                           |
| Created         | 2026-06-10                                                                                                                                                                                                                  |
| Last modified   | 2026-06-10                                                                                                                                                                                                                  |
| Status          | active                                                                                                                                                                                                                      |
| Status summary  | DESIGN — read-only over the repo. NO code edited, NO repo created, NO network/ mutating git performed. EVERY repo-creation recommendation is OPERATOR-GATED (§11.4.101 — irreversible/high-blast-radius external mutation). |
| Source research | docs/research/2026-06-10-execution-models.md (SP6 feeder), docs/research/2026-06-10-parallel-agent-coordination.md (SP5 feeder)                                                                                             |
| Source plan     | docs/superpowers/specs/plans/2026-06-10-SP5-SP6-parallel-dynamic-plan.md                                                                                                                                                    |
| Authority       | Cascades from constitution/ + root CLAUDE.md/CONSTITUTION.md. Anti-bluff §11.4 family + §11.4.74 catalogue-first + CONST-051/054 govern every decision.                                                                     |

**Anti-bluff note (§11.4.6 / §11.4.123):** This is a DESIGN, not a claim of done work. No interface below is implemented. Every in-repo fact carries a real `file:line` opened this session; every “absent” is stated explicitly; every external claim is sourced from the two research docs (which themselves carry `## Sources verified` footers). Items that could not be confirmed are flagged HONEST-GAP rather than asserted. The purpose of this doc is solely to give the operator enough detail to make the **G-2 decision** (which new repos to create) with full scope.

---

### Table of contents

- [0. Grounding — verified facts \(file:line\)](#)
- [1. Decision frame: CREATE vs EXTEND vs REUSE vs DEFER](#)
- [2. The shared cross-cutting contracts \(defined once, consumed by all\)](#)

- [3. Per-proposed-repo design](#)
    - [3.1 dag\\_orchestrator — dev.helix.dag — CREATE](#)
    - [3.2 pipeline\\_runtime — dev.helix.pipeline — CREATE](#)
    - [3.3 flow\\_engine — dev.helix.flow — EXTEND Agentic \(CREATE only if it can't host\)](#)
    - [3.4 saga — fold into flow\\_engine / wrap Temporal — DO NOT create a repo](#)
    - [3.5 behavior\\_tree — dev.helix.bt — DEFER \(small CREATE later\)](#)
  - [4. helix-deps.yaml + flat-layout placement \(CONST-054 / CONST-051\)](#)
  - [5. Refined G-2 recommendation \(create-now / extend / defer\)](#)
  - [6. Ten-line summary](#)
- 

## 0. Grounding — verified facts (file:line)

Every anchor below was opened READ-ONLY this session.

**Reuse baseline (what exists today — §11.4.74 catalogue):** - submodules/agentica/agentica/workflow.go:48-90 — WorkflowGraph{Nodes map[string]\*Node, Edges []\*Edge, EntryPoint, EndNodes}; Node{ID, Name, Type, Handler, Condition, Config, RetryPolicy}; NodeType ∈ {agent, tool, condition, parallel, human, subgraph} (workflow.go:67-75); Edge{From, To, Condition, Label}; NodeHandler func(ctx, \*WorkflowState, \*NodeInput) (\*NodeOutput, error); WorkflowState at workflow.go:140. Module digital.vasic.agentica, files under the agentica/agentica/ subdir (not repo top level). - submodules/agentica/agentica/workflow.go:240,253 — Workflow.AddNode(\*Node) + Workflow.AddEdge(from, to, cond, label) **DO exist and are w. mu-guarded.** HONEST-CORRECTION to the SP6 research line “no runtime AddNode/AddEdge/mutate API found”: the *methods* exist (builder-pattern). The substantive gap is that a NodeHandler returns only \*NodeOutput — it **cannot name its successors or spawn new nodes** (no LangGraph Command/Send equivalent), and there is no documented during-Run() re-planning contract. So “runtime-mutable topology driven by a node’s own result” is the real gap, not the literal absence of AddNode. - submodules/concurrency/pkg/pool/pool.go:13-17 — type Task interface { ID() string; Execute(ctx) (interface{}, error) } (the minimal dispatchable-unit interface). pool.go:567 — func Map[T any, R any] (...) parallel-map. WorkerPool + Submit/SubmitWait confirmed via pool\_edge\_test.go. - submodules/concurrency/pkg/queue/queue.go:14-17 — priority levels Low=0 ... Critical=3; queue.go:92 — func New[T any](initialCap int) \*PriorityQueue[T] (generic). Module digital.vasic.concurrency. - helix\_code/internal/task/dependency.go:19,46 — DependencyManager.ValidateDependencies([]uuid.UUID) error + CheckDependenciesCompleted([]uuid.UUID) (bool, error). This is **dependency DATA + completion gating, not a scheduler.** - helix\_code/internal/workflow/executor.go + executor\_test.go — a **linear** step runner (ExecutePlanningWorkflow/ExecuteBuildingWorkflow/ExecuteStep\_\*/ExecuteBuildStep\_Go|Node|Python|Rust), NOT a general DAG scheduler. - .gitmodules — own-org submodules already cloned + wired: submodules/agentica → git@github.com:vasic-digital/Agentica.git; submodules/concurrency → vasic-digital/Concurrency.git; submodules/planning → vasic-digital/Planning.git; submodules/llm\_orchestrator → HelixDevelopment/LLMOrchestrator.git; submodules/debate\_orchestrator → HelixDevelopment/DebateOrchestrator.git;

submodules/helix\_agent → HelixDevelopment/HelixAgent.git;  
submodules/event\_bus, submodules/streaming present. - helix\_code/go.mod  
— grep -nE 'agentic|concurrency|planning|dev.helix.dag|  
dev.helix.pipeline' → **EMPTY** (confirmed this pass). HelixCode does NOT consume  
the own-org agentic/concurrency/planning substrate today — it re-implements coordination in  
internal/{agent, worker, workflow, task}. This is the D-7 gap and the reason  
CREATE/EXTEND must be paired with a replace-wiring step.

**helix-deps.yaml schema (verified submodules/security/helix-deps.yaml):**  
schema\_version: 1; deps: [...]; transitive\_handling.{recursive,  
conflict\_resolution}; language\_specific\_subtree.submodules/  
agentic/helix-deps.yaml is **absent/empty** (HONEST-GAP — Agentic has no manifest  
yet; any EXTEND of Agentic should add one as part of the work).

---

## 1. Decision frame: CREATE vs EXTEND vs REUSE vs DEFER

The research recommended at minimum dag\_orchestrator + pipeline\_runtime, and  
floated flow\_engine, saga, behavior\_tree. The §11.4.74 ruling for each:

| Family<br>(research §)                                                                      | What's genuinely<br>absent today                                                                                                                                                                                                                                                                           | Closest existing home                                      | Verdict                           |
|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|-----------------------------------|
| <b>DAG<br/>scheduling</b><br>(exec §6)                                                      | A <b>scheduler</b><br>(ready-set dispatch<br>+ parallelism cap +<br>failure policy +<br>lineage). We have<br>dependency DATA<br>(task/<br>dependency.go)<br>+ a linear runner<br>(workflow/<br>executor.go),<br>no scheduler. No<br>dominant Go DAG-<br>scheduler lib exists<br>(research HONEST-<br>GAP). | none — concurrency is<br>the substrate, not a<br>scheduler | <b>CREATE</b><br>dag_orchestrator |
| <b>Dataflow/<br/>reactive +<br/>pipeline/<br/>map-reduce</b><br>(exec §2,§5;<br>parallel §) | Push-based<br>streaming operators<br>(Map/Filter/<br>FlatMap/Window)<br>+ typed-port FBP<br>wiring +<br>backpressure<br>ergonomics.<br>submodules/<br>streaming is<br>LLM token-<br>streaming, NOT a<br>dataflow operator                                                                                  | none                                                       | <b>CREATE</b><br>pipeline_runtime |

| Family<br>(research §)                                 | What's genuinely<br>absent today                                                                                                                                                                                                               | Closest existing home                                                     | Verdict                                                                                        |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
|                                                        | graph (verify-before-reuse<br>flagged in research).                                                                                                                                                                                            |                                                                           |                                                                                                |
| <b>Dynamic flows / runtime-mutable graph</b> (exec §1) | Node-result-driven routing (Command), dynamic fan-out (Send), during-run re-planning. <code>agent ic</code> covers typed nodes + conditional/parallel/subgraph edges ( $\geq 80\%$ ) but <code>NodeHandler</code> can't name successors/spawn. | <code>vasic-digital/Agent ic</code><br>( <code>workflow.go:48-90</code> ) | <b>EXTEND Agent ic</b><br>(new repo only if dynamic re-plan can't land in Agent ic)            |
| <b>Saga / compensation</b> (exec §3)                   | Explicit compensation/rollback (Do/Undo, reverse-order). We have retry+checkpoint, no compensation model. Crash-durable sagas → wrap <code>temporal io/sdk-go</code> (Go, MIT), don't reimplement.                                             | belongs inside <code>flow_engine</code> (the extended Agent ic)           | <b>FOLD — not a repo</b>                                                                       |
| <b>Behavior trees</b> (exec §4)                        | Tick-based reactive control (Success/Failure/ <b>Running</b> ) + Selector fallbacks. <code>agent ic</code> is run-to-completion, no per-tick re-eval. Small, vendorable from <code>askft/go-behave</code> (MIT).                               | none, but small + lower-priority                                          | <b>DEFER</b> (small <code>CREATE behavior_tree</code> later)                                   |
| <b>Planning/search</b> (parallel §)                    | nothing — <code>vasic-digital/Planning</code> already ships HiPlan/MCTS/ToT at                                                                                                                                                                 | <code>vasic-digital/Planning</code>                                       | <b>REUSE</b> (no repo; HelixCode binding only — already covered in the plan, out of G-2 scope) |

| Family<br>(research §)                     | What's genuinely<br>absent today      | Closest existing home                                                       | Verdict                                                                  |
|--------------------------------------------|---------------------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------|
|                                            | submodules/<br>planning               |                                                                             |                                                                          |
| <b>Agent-mesh /<br/>swarm</b> (exec<br>§7) | Swarm handoff +<br>supervisor routing | debate_orchestrator<br>+ llm_orchestrator +<br>helix_agent agents/<br>swarm | <b>EXTEND/FOLD into<br/>SP5 substrate</b> (out of<br>G-2 new-repo scope) |

**Net G-2 scope:** two genuinely-new repos warrant creation NOW — dag\_orchestrator and pipeline\_runtime. flow\_engine is an EXTEND of Agentic (repo only as a fallback). saga folds into flow\_engine. behavior\_tree defers. The smallest set that delivers value is therefore **2 new repos**.

## 2. The shared cross-cutting contracts (defined once, consumed by all)

Per research exec §8, all execution-model families must compose on **one** scheduling substrate (concurrency) and **one** state model (agentic.WorkflowState). To avoid each new repo inventing its own Executor/Result, the contracts below live in the **SP5 shared substrate** (the Unit/Scheduler/Dispatcher/Resolver module — extend Concurrency per plan OD-1; out of G-2 scope but the new SP6 repos DEPEND on it). The new SP6 repos implement / consume these — they do NOT redefine them:

```
// Owned by the SP5 substrate (extend digital.vasic.concurrency,
//   or dev.helix.substrate).
// SP6 repos import these — single source of truth, no
//   duplication (§11.4.74).
```

```
type Unit interface {           // a self-describing
    dispatchable piece of work
    ID() string                 // (already = concurrency
    pool.Task.ID — pool.go:15)
    Execute(ctx context.Context) (interface{}, error) // (=
    pool.Task.Execute — pool.go:17)
}
```

```
type Scheduler interface
{
    // decides WHICH ready units run now, honoring
    parallelism + priority
    Submit(u Unit) Handle
    Await(h Handle) (Result, error)
    Parallelism() int
}
```



```

    Execute(ctx context.Context, in Inputs) (Output, error)
}

type FailurePolicy int
const ( FailFast FailurePolicy = iota; ContinueOnError )

type DAG struct{ /* nodes map[string]Node; validated acyclic at
    Build() */ }
func Build(nodes []Node) (*DAG, error) // validates acyclicity
    (cycle => error)

type Scheduler interface {
    // Run dispatches ready nodes onto the injected
    // concurrency.Scheduler honoring
    // Parallelism + Priority, applies RetryPolicy +
    // FailurePolicy, returns lineage.
    Run(ctx context.Context, d *DAG, opts Options) (*Result,
        error)
}

type Options struct {
    Parallelism    int
    Failure        FailurePolicy
    Retry          RetryPolicy
    Priority        func(Node) queue.Priority // maps to
        concurrency queue.Critical..Low
}

type Result struct {
    Outputs map[string]Output // per-node
    Lineage []Edge           // who-ran-after-whom (Dagster-
        style)
    Failed  map[string]error
}

// Dynamic DAG (bridges to flow_engine §1 Send): a completed node
// may emit new nodes.
type Expandable interface{ Expand(ctx context.Context, out
    Output) ([]Node, error) }

```

**Wiring points.** - **HelixCode** — `helix_code/internal/workflow/` consumes `dag.Scheduler` for non-agent DAGs (replaces the linear executor `.go` path for branching/parallel work); `helix_code/internal/task` provides the dependency data (`DependencyManager` → `Node.DependsOn`). Requires the D-7 replace `dev.helix.dag => ../submodules/dag_orchestrator` + `require` in `helix_code/go.mod` (currently absent). - **HelixAgent** — `submodules/helix_agent/internal/agent` uses `dag.Scheduler` to schedule its tool-graph nodes; keeps the public `agent` API stable (thin adapter, CONST-051(B) config-injection). - **Substrate** — internally calls the SP5 `Scheduler` (or `concurrency.pool.WorkerPool` +

queue.PriorityQueue + semaphore directly) to run ready nodes. No reimplementa-  
tion of the pool.

**helix-deps.yaml (CONST-054).** See §4.

**Anti-bluff Challenge that proves it works (§11.4.5/§11.4.69/§11.4.135).** challenges/  
banks/sp6-dag-orchestrator-challenge.sh — build a real diamond DAG  $A \rightarrow \{B, C\} \rightarrow D$  where B sleeps and C is fast; assert from captured stdout that (1) **B and C ran concurrently** (overlapping start/end timestamps captured to an evidence file — not a single timestamp), (2) **D ran strictly after both B and C completed** (ordering proof), (3) under FailFast a planted failure in B **prevents D from starting** and **cancels C** with a captured cancellation log, (4) under ContinueOnError D still runs and Result.Failed[B] is populated. Paired §1.1 mutation: break the ready-set gate so D dispatches before B/C finish → the ordering assertion FAILs. Evidence class boot\_service/runtime-signature per §11.4.69; transcript under docs/qa/<run-id> ( §11.4.83). RED-on-broken-artifact polarity switch per §11.4.115.

---

### 3.2 pipeline\_runtime — dev.helix.pipeline — CREATE NOW

| Property    | Value                                                                                                                                         |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Repo name   | PipelineRuntime → submodules/<br>pipeline_runtime/                                                                                            |
| Module path | dev.helix.pipeline                                                                                                                            |
| Org         | <b>HelixDevelopment</b>                                                                                                                       |
| Visibility  | <b>public</b>                                                                                                                                 |
| Verdict     | <b>CREATE</b> — push-based streaming + FBP<br>wiring absent; submodules/streaming<br>is LLM-token streaming, not a dataflow<br>operator graph |

**Purpose + what’s genuinely absent.** A staged streaming dataflow runtime: composable push-based **operators** (Map/Filter/FlatMap/Window/Buffer/Reduce) over typed channels, plus a **Component/Port** flow-based-programming (FBP) layer for declaratively-wired agent pipelines, plus a **map-reduce** helper (partition → map → shuffle → reduce) and Unix-pipe-style stage composition. Backpressure reuses concurrency’s rate-limiter/semaphore. The research (exec §2, §5; parallel §) found this entirely absent: submodules/streaming is LLM token streaming (verify-before-reuse), and Go channels + concurrency.pool.Map cover only the low-level substrate, not the ergonomic operator/FBP API. Recommendation is an **Rx + FBP subset first** (RxGo/GoFlow patterns, both Go/MIT) — NOT full Apache Beam parity (HONEST-GAP: Beam parity is large and over-scoped).

**Core Go interfaces it exposes.**

```
package pipeline // dev.helix.pipeline

type Stream[T any] interface
    {
        // push-based observable (RxGo-shaped)
        Subscribe(ctx context.Context, sink func(T) error) error
    }
```



```

type Operator[T any, U any] func(in Stream[T]) Stream[U]    //
    composable stage

// Operator constructors:
func Map[T, U any](fn func(T) (U, error)) Operator[T, U]
func Filter[T any](pred func(T) bool) Operator[T, T]
func FlatMap[T, U any](fn func(T) []U) Operator[T, U]
func Window[T any](size int, every time.Duration) Operator[T,
    []T]
func MapReduce[T, K comparable, R any](
    mapper func(T) (K, R), reducer func(K, []R) R) Operator[T, R]

// FBP layer: named typed-port components wired into a network.
type Component interface {
    In() []Port; Out() []Port
    Run(ctx context.Context) error
}
type Network struct{ /* components + channel wiring */ }
func (n *Network) Connect(from Port, to Port) error
func (n *Network) Run(ctx context.Context) error

// Backpressure injected from concurrency (semaphore / token-
    bucket) – no reimplementation.
type Backpressure interface{ Acquire(ctx context.Context) error;
    Release() }

```

**Wiring points. - HelixCode** — `helix_code/internal/repomap + index-build` use a `pipeline.Network` (`scan` → `parse` → `embed` → `write` stages); the LLM streaming path can expose a `pipeline.Stream[Token]`. Requires `replace dev.helix.pipeline => ../submodules/pipeline_runtime` in `helix_code/go.mod`. - **HelixAgent** — RAG ingest (`chunk` → `embed` → `upsert`) wires as an FBP `Network`; reuses `submodules/concurrency` for backpressure. - **Substrate** — `MapReduce` and parallel `Map` fan-out over `concurrency.pool.Map` (`pool.go:567`); bounded buffers + semaphore for backpressure.

**helix-deps.yaml (CONST-054)**. See §4.

**Anti-bluff Challenge (§11.4.5/§11.4.69/§11.4.135)**. `challenges/banks/sp6-pipeline-runtime-challenge.sh` — feed a real bounded source of 10,000 records through `Map` → `Filter` → `Window(100)` → `MapReduce` and assert from captured output: (1) the reducer’s emitted aggregate **matches an independently-computed golden value** (full-reference correctness, not “it ran”), (2) under a slow sink the source **blocks via backpressure** rather than unbounded-buffering — captured by an in-flight-count probe staying  $\leq$  the semaphore weight (not OOM), (3) **no goroutine leak** after completion (goroutine count returns to baseline — reuse concurrency/deadlock or `runtime.NumGoroutine` delta), (4) cancellation mid-stream stops all stages cleanly (captured shutdown log). Paired §1.1 mutation: remove the backpressure `Acquire` → the in-flight-count assertion FAILs (unbounded buffering detected). §11.4.85 stress (sustained 30 s) + chaos (kill a stage mid-stream → clean refuse). Evidence under `docs/qa/<run-id>/`.

---

### 3.3 flow\_engine — dev.helix.flow — EXTEND Agentic first

| Property                    | Value                                                                                                                                                              |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Repo name (fallback only)   | FlowEngine → submodules/<br>flow_engine/                                                                                                                           |
| Module path (fallback only) | dev.helix.flow                                                                                                                                                     |
| Org                         | HelixDevelopment                                                                                                                                                   |
| Visibility                  | public                                                                                                                                                             |
| Verdict                     | <b>EXTEND</b> vasic-digital/Agentic —<br>CREATE a new repo only if dynamic re-<br>planning genuinely can't land in Agentic<br>(operator decision; default: extend) |

**Purpose + why EXTEND not CREATE.** Dynamic flows = runtime-mutable graph topology driven by a node's own result: a node returns *state-patch* + *next-node(s)* in one value (LangGraph Command), a conditional edge dynamically invokes a node N times with per-branch state (LangGraph Send, native map-reduce), and during-run re-planning. The research (exec §1) measures this against agentic: WorkflowGraph/Node/Edge/NodeType{agent, tool, condition, parallel, human, subgraph} already cover ≥80% of the surface (workflow.go:48-90), and AddNode/AddEdge already exist mutex-guarded (workflow.go:240,253). The genuine gap is narrow: NodeHandler returns only \*NodeOutput — it **cannot name successors or spawn nodes**. Adding a Command/Send result shape to the *existing* engine is a far smaller, lower-risk change than standing up a parallel flow engine (which would duplicate the whole graph/state/checkpoint apparatus — a §11.4.74 violation). Therefore: **extend Agentic in-place** (PR upstream, bump pointer), and reserve a new flow\_engine repo only as a fallback if the operator finds dynamic re-planning architecturally incompatible with Agentic.

**Extension to land in Agentic (not a new interface set — an addition to the existing one).**

```
// ADD to digital.vasic.agentic (extend workflow.go's NodeHandler
// result):
type Command struct
{
    // LangGraph Command — patch + routing
    // in one value
    StatePatch map[string]interface{}
    GoTo       []string           // successor node IDs
               (overrides static edges)
    Spawn      []*Node        // dynamic nodes added at
               runtime (Send / map-reduce)
}
// NodeHandler gains an optional Command return (additive, back-
// compat):
type RoutingHandler func(ctx, *WorkflowState, *NodeInput)
(*NodeOutput, *Command, error)
```

```
// Fan: dynamic map-reduce over concurrency.pool.Map (LangGraph
    Send).
func (w *Workflow) Fan(target string, states []*WorkflowState)
    ([]*NodeOutput, error)
```

Also lands (research exec §1, §8): a reducer-typed `WorkflowState` merge, and checkpoints copying **messages+history** not just variables (the §4 gap flagged in the SP5 research). Durable replay is OUT — research is explicit: **do not reimplement Temporal**; wrap `temporalio/sdk-go` (Go, MIT) behind an optional `Durability` adapter only where crash-durability is required (§3.4).

**Wiring points.** `helix_code/internal/workflow/executor.go` registers flow tools and runs branching workflows through the extended `Agentic`; `submodules/helix_agent/internal/agent` already IS `Agentic`-shaped, so it consumes the new `Command/Send` directly. Requires replace `dev.helix.agentic`-equivalent wiring into `helix_code/go.mod` (D-7 — currently absent for `agentic`).

**helix-deps.yaml.** If `EXTEND`: add `helix-deps.yaml` to `submodules/agent` (currently absent — `HONEST-GAP`) declaring its concurrency dep. If the fallback new repo is created: see §4.

**Anti-bluff Challenge.** `challenges/banks/sp6-flow-engine-challenge.sh` — a real flow where `node route` returns `Command{GoTo: [...]}` chosen from runtime state (assert the captured execution path matches the state-driven branch, and a control run with different state takes a *different* captured path — metamorphic relation), and a `fanout` node Spawns `N` sub-nodes via `Send` whose results are gathered (assert all `N` captured outputs present + order-independent merge). Paired §1.1 mutation: ignore the `Command.GoTo` (fall back to static edge) → the path-match assertion FAILs. Evidence under `docs/qa/<run-id>/`.

### 3.4 saga — FOLD, not a new repo

|         |                                                                                                                                             |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Verdict | <b>FOLD into <code>flow_engine</code> (the extended <code>Agentic</code>); REUSE <code>temporalio/sdk-go</code> for crash-durable sagas</b> |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------|

**Why no repo.** A saga is a thin primitive: a sequence of steps each with a registered compensation, run forward collecting `Undo` closures, on error run them LIFO. Research (exec §3) is explicit: the **lightweight** in-process saga is a ~one-file primitive that belongs *inside* `flow_engine`, and **crash-durable** sagas must wrap `temporalio/sdk-go` (Go, MIT, first-class saga support — `temporalio/samples-go/saga/workflow.go`) rather than reimplement durable event-sourcing. A standalone saga repo would be too small to justify the submodule overhead and would duplicate `Temporal`'s durable layer. The contract:

```
// inside dev.helix.flow (or the extended agent):
type Compensable interface{ Do(ctx context.Context) error;
    Undo(ctx context.Context) error }
type Saga struct{ steps []Compensable }
func (s *Saga) Run(ctx context.Context)
    error // forward; on err runs Undo in reverse
```

**Wiring.** `flow_engine` nodes opt into saga semantics; durable variant injected via the optional `Temporal Durability` adapter. **Challenge** lives with `flow_engine`: a 3-step saga where step 3

fails → assert steps 2 and 1 Undo ran in reverse order (captured compensation log) + a paired mutation that runs Undo forward-order → FAIL.

---

### 3.5 behavior\_tree — dev.helix.bt — DEFER

| Property            | Value                                                                                    |
|---------------------|------------------------------------------------------------------------------------------|
| Repo name (later)   | BehaviorTree → submodules/<br>behavior_tree/                                             |
| Module path (later) | dev.helix.bt                                                                             |
| Org                 | HelixDevelopment                                                                         |
| Visibility          | public                                                                                   |
| Verdict             | <b>DEFER</b> — small, lower-priority CREATE later; vendor askft/go-behave (MIT) patterns |

**Why defer (not never, not now).** Behavior trees add genuinely-distinct value the current model lacks: tick-based reactive control returning Success/Failure/**Running**, Selector (OR-fallback) and Sequence (AND), decorators (Inverter/Retry/Timeout/Repeat), and a blackboard — *agentic* is run-to-completion graph traversal with no per-tick re-evaluation (research exec §4). BUT: (1) it is **not on the critical path** — *dag\_orchestrator* + *pipeline\_runtime* + *flow\_engine* cover the dominant execution models the operator mandate names; (2) it is **small and self-contained** (vendorable from askft/go-behave, MIT) so it can be created cheaply later with no rework; (3) the research flags a **license HONEST-GAP** on the alternative *joeycumines/go-behaviortree* (SPDX unconfirmed) — confirm the LICENSE before vendoring. Deferring keeps the G-2 create-now set minimal while preserving the option. When created, BT **leaves delegate to** *agentic/llm\_orchestrator* handlers (BT becomes a control layer over existing handlers, not a new execution substrate), and the blackboard aliases *agentic.WorkflowState.Variables*.

**Contract (for the later CREATE).**

```
package bt // dev.helix.bt
type Status int
const ( Success Status = iota; Failure; Running )
type Tick func(ctx context.Context) (Status, error)
func Sequence(children ...Tick) Tick // AND
func Selector(children ...Tick) Tick // OR / fallback
func Parallel(children ...Tick) Tick
// decorators: Inverter / Retry(n) / Timeout(d) / Repeat(n)
```

**Challenge (later).** A tree whose Selector falls back from a failing primary leaf to a succeeding secondary leaf → assert the captured tick trace shows primary→Failure then secondary→Success then root→Success; paired mutation swapping Selector for Sequence → root FAILs.

---

## 4. helix-deps.yaml + flat-layout placement

**Flat layout (CONST-051(C)).** Every new repo is cloned at the project root as a flat sibling — `<repo-root>/submodules/<name>/` — matching the existing convention (submodules/agentic/, submodules/concurrency/, ...). **NO nested own-org submodule chains** (CONST-051(C)): a new repo's own-org deps (e.g. concurrency) are reached from the HelixCode root, never via the new repo's own nested .gitmodules. .gitmodules entries use **SSH only** (Rule 3 / no-HTTPS).

submodules/dag\_orchestrator/helix-deps.yaml:

```
# helix-deps.yaml — DagOrchestrator Submodule-Dependency Manifest
# (CONST-054 / §11.4.31)
schema_version: 1
deps:
  - name: concurrency
    ssh_url: git@github.com:vasic-digital/Concurrency.git
    ref: main # operator pins a SHA at incorporation
              per §11.4.74
    why: "worker pool + priority queue + semaphore — the
          scheduling substrate the DAG dispatches onto"
    layout: flat
  # OPTIONAL (only if SP5 substrate repo is created at OD-1):
  # - name: agent_substrate
  #   ssh_url: git@github.com:HelixDevelopment/AgentSubstrate.git
  #   ref: main
  #   why: "shared Unit/Scheduler contract the DAG nodes
  #         implement"
  #   layout: flat
transitive_handling:
  recursive: true
  conflict_resolution: operator-required
language_specific_subtree: false
```

submodules/pipeline\_runtime/helix-deps.yaml:

```
# helix-deps.yaml — PipelineRuntime Submodule-Dependency Manifest
# (CONST-054 / §11.4.31)
schema_version: 1
deps:
  - name: concurrency
    ssh_url: git@github.com:vasic-digital/Concurrency.git
    ref: main
    why: "pool.Map fan-out + semaphore/rate-limiter backpressure
          for streaming stages"
    layout: flat
transitive_handling:
  recursive: true
```

```
    conflict_resolution: operator-required
language_specific_subtree: false

submodules/flow_engine/helix-deps.yaml (fallback-only — if NOT extending
Agentic):
```

```
schema_version: 1
deps:
  - name: agentic
    ssh_url: git@github.com:vasic-digital/Agentic.git
    ref: main
    why: "graph/node/state engine flow_engine builds dynamic-
        routing on top of"
    layout: flat
  - name: concurrency
    ssh_url: git@github.com:vasic-digital/Concurrency.git
    ref: main
    why: "Fan/Send map-reduce fan-out substrate"
    layout: flat
transitive_handling:
  recursive: true
  conflict_resolution: operator-required
language_specific_subtree: false
```

Default path (EXTEND Agentic) instead adds a `helix-deps.yaml` to `submodules/agentic` (currently absent) declaring its concurrency dep — no new repo, no new `.gitmodules` entry.

Each new repo also ships its **OWN** standalone full test matrix + Challenges (CONST-050(B), CONST-051 standalone-testable), and the §11.4.135 regression guard is registered per closure. Each is paired with an `incorporate-submodule` Challenge (CONST-054) that bootstraps a throwaway consuming project, runs `incorporate-submodule <ssh-url>`, and asserts the produced layout matches the manifest.

---

## 5. Refined G-2 recommendation

The minimal set that delivers value. Every CREATE is **OPERATOR-GATED** (§11.4.101 — repo creation is irreversible external mutation; perform only on explicit operator approval, SSH remotes only, no force-push §11.4.113).

| #    | Repo             | Module             | Org / vis                 | Verdict                         |
|------|------------------|--------------------|---------------------------|---------------------------------|
| G2-1 | dag_orchestrator | dev.helix.dag      | HelixDevelopment / public | <b>CREATE NOW</b>               |
| G2-2 | pipeline_runtime | dev.helix.pipeline | HelixDevelopment / public | <b>CREATE NOW</b>               |
| G2-3 | flow_engine      | dev.helix.flow     | HelixDevelopment / public | <b>EXTEND</b><br>vasic-digital/ |

| #    | Repo             | Module                 | Org / vis                 | Verdict                                                                             |
|------|------------------|------------------------|---------------------------|-------------------------------------------------------------------------------------|
|      |                  |                        |                           | Agentic (CREATE repo only as fallback if dynamic re-plan can't land in Agentic)     |
| G2-4 | saga             | —                      | —                         | <b>FOLD into flow_engine</b> + REUSE temporalio/sdk-go for durable — <b>NO repo</b> |
| G2-5 | behavior_tree    | dev.helix.bt           | HelixDevelopment / public | <b>DEFER</b> (small CREATE later; confirm vendor LICENSE first)                     |
| —    | planning_search  | digital.vasic.planning | (existing)                | <b>REUSE</b> vasic-digital/Planning — NO repo (out of G-2 scope; plan-tracked)      |
| —    | agent_mesh/swarm | —                      | —                         | <b>EXTEND/ FOLD into SP5 substrate</b> — NO new G-2 repo (out of scope)             |

**Net new public repos warranted at G-2: TWO** — dag\_orchestrator + pipeline\_runtime. Everything else is EXTEND, FOLD, REUSE, or DEFER. This matches and tightens the plan's §6.12 recommendation, with the added precision that saga is explicitly a fold (not even a deferred repo) and behavior\_tree is the single deferred candidate.

## 6. Ten-line summary

1. **Scope of G-2:** decide which NEW HelixDevelopment repos to create for SP6's execution-model families; this design gives full per-repo detail so the operator can decide precisely.
2. **CREATE NOW (2 repos):** dag\_orchestrator (dev.helix.dag) — a real DAG scheduler (we have dep-data task/dependency.go:19,46 + a linear runner workflow/executor.go, no scheduler); pipeline\_runtime (dev.helix.pipeline) — push-based streaming + FBP (absent; streaming is token-streaming, not dataflow).
3. **EXTEND not create:** flow\_engine → extend vasic-digital/Agentic — workflow.go:48-90 already covers ≥80%; the narrow gap is a Command/Send (GoTo/Spawn) result shape on NodeHandler. New repo only as fallback.

4. **FOLD not a repo:** saga is a one-file Compensable{Do,Undo} primitive inside flow\_engine; crash-durable sagas wrap temporalio/sdk-go (Go, MIT) — never reimplement Temporal.
  5. **DEFER:** behavior\_tree (dev.helix.bt) — genuinely distinct (tick/Running/Selector) but off the critical path and small; confirm joeycumines/go-behaviortree LICENSE (HONEST-GAP) before vendoring; default vendor askft/go-behave (MIT).
  6. **One substrate, no duplication (§11.4.74):** both new repos consume the SP5 Scheduler (or concurrency.pool+queue+semaphore directly — pool.go:13-17, pool.go:567, queue.go:14-17); they add only their narrow DAG/Stream contracts.
  7. **CONST-051/054:** each new repo is flat at submodules/<name>/, SSH-only, project-not-aware, standalone-testable, ships helix-deps.yaml (schema verified vs submodules/security/helix-deps.yaml) declaring its concurrency dep with no nested own-org chains.
  8. **D-7 wiring is mandatory:** helix\_code/go.mod currently has NO own-org agentic/concurrency/dag/pipeline require/replace (grep empty) — every CREATE/EXTEND must be paired with the replace ... => ../submodules/<name> wiring or the repo sits unused.
  9. **Anti-bluff Challenges (§11.4.5/§11.4.69/§11.4.135):** each repo's Challenge proves real behavior with captured evidence + a paired §1.1 mutation (DAG: concurrent-then-ordered diamond; pipeline: golden-value + backpressure + no-leak; flow: state-driven path metamorphic; saga: reverse-order compensation).
  10. **Refined G-2 verdict:** create exactly **two** repos now — dag\_orchestrator + pipeline\_runtime — EXTEND Agentic for flow\_engine, FOLD saga, DEFER behavior\_tree, REUSE Planning; every CREATE is OPERATOR-GATED (§11.4.101).
- 

## Sources verified 2026-06-10

In-repo facts cited from real file:line opened this session (see §0). External execution-model claims (LangGraph Command/Send, Temporal Go SDK saga, RxGo/GoFlow, behavior-tree libs, OpenAI Agents SDK handoff, DAG-orchestrator landscape) are sourced from the two completed research docs, which carry their own ## Sources verified 2026-06-10 footers: - docs/research/2026-06-10-execution-models.md (§0-§9 + Sources footer) - docs/research/2026-06-10-parallel-agent-coordination.md (§1-§8 + Sources footer)

**Honest gaps / corrections recorded in this design:** (a) agentic.Workflow.AddNode/AddEdge DO exist (workflow.go:240,253) — the real dynamic-flow gap is Command/Send (GoTo/Spawn) on NodeHandler, not literal absence of mutation methods. (b) submodules/agentic/helix-deps.yaml is absent — an EXTEND must add it. (c) submodules/streaming scope (LLM token vs dataflow) should be confirmed before any reuse — assumed token-streaming per research. (d) joeycumines/go-behaviortree SPDX license unconfirmed — verify before vendoring (behavior\_tree is deferred regardless). (e) full Apache Beam parity is out of scope for pipeline\_runtime — Rx+FBP subset only.